

myRekod — Developer Handover Guide

For engineers taking over this project

myRekod documentation

Start here if you are a new **user** or a new **developer** taking over this project.

myRekod is a Flutter HR app for:

- Attendance (clock in / out)
 - Leave
 - Expense claims
 - Payroll / payslips
 - Announcements
 - Workplace location (geofence for clock-in)
-

Who should read what?

I am a staff member (employee)

Read these in order:

1. [Getting started](#)
2. [Employee guide](#)
3. [Troubleshooting](#) — only when something goes wrong

I am HR / an administrator

Read these in order:

1. [Getting started](#)
2. [Admin guide](#)
3. [Employee guide](#) — useful so you know what staff see
4. [Troubleshooting](#)

I am a new developer

Read these in order:

1. [Project overview](#)
2. [Setup and run](#)
3. [Supabase and database](#)
4. [Architecture](#)
5. [Features and key files](#)
6. [Deploy and auth URLs](#)
7. [Handover checklist](#)

Also skim the **user** guides so you understand the product.

Folder map

docs/	
README.md	← you are here
user/	← for employees & admins (non-technical)
developer/	← for engineers taking over the codebase

Support contact (in-app)

Item	Value
Phone	+60 11-7078 7014
Hours	Monday–Friday, 9:00–18:00 Malaysia Time (MYT)

Configured in: `lib/constants/help_support_config.dart`

Quick product facts

Topic	Fact
App name	myRekod
Platform	Flutter (web first; also Android/iOS capable)
Backend	Supabase (Auth, Postgres, Storage, Realtime)
Claims currency	MYR only
Geofence	One workplace site; clock-in only
Password reset	Email link → set new password → login

Keep this folder updated when features change.

Developer overview

This document is for a **new developer** taking over myRekod.

What you are inheriting

Item	Detail
Product	myRekod — attendance, leave, claims, payroll, announcements, geofence
Client	Flutter app (attendance_app)
Backend	Supabase (Auth, Postgres, Storage, Realtime)
Primary UI target	Web (Chrome / Safari PWA), also runnable on Android/iOS/desktop
State	Provider (AuthProvider)

Repository layout (important folders)

attendance_app/	
lib/	Flutter app source
config/app_config.dart	Supabase URL + anon key defaults / env
constants/	Theme, help support config
models/	Data models
providers/	AuthProvider
screens/	UI (admin/ + employee/)
services/	SupabaseService, payroll, realtime
utils/	Routes, time, geofence, validators
widgets/	Shared UI (OSM map, tiles, etc.)
supabase_bundles/	Ordered SQL for NEW projects
supabase_migration_*.sql	One-off migrations for EXISTING projects
docs/	This handover documentation
pubspec.yaml	Dependencies

Main product areas

1. **Auth** — email/password, username login, password recovery UI
2. **Attendance** — clock in/out, history, admin overview
3. **Geofence** — one `work_site` , clock-in only
4. **Leave** — apply, approve, attachments, balances
5. **Claims** — MYR expenses + attachments
6. **Payroll** — salary settings, statutory, runs, payslips
7. **Announcements** — company posts + unread badge
8. **Help** — FAQs + support phone

Roles

Stored in `public.users.role` :

- `employee` → `EmployeeShell`
- `admin` → `AdminShell`

Promote an admin in SQL:

```
UPDATE public.users
SET role = 'admin'
WHERE email = 'someone@company.com';
```

Documentation map for developers

File	Purpose
------	---------

01 setup and run.md	Install, run, env
02 supabase and database.md	SQL bundles & migrations
03 architecture.md	App structure & patterns
04 features and key files.md	Where each feature lives
05 deploy and auth.md	Vercel + Auth redirect URLs
06 handover checklist.md	Day-1 checklist

Also read user docs under `docs/user/` so you understand the product.

Related SQL docs

- Bundle guide: [supabase bundles/README.md](#)
 - Work site migration: [supabase migration work site geofence.sql](#)
-

Setup and run

Follow this to run myRekod on your machine.

Prerequisites

- Flutter SDK (project uses Dart SDK ^3.8.1)
- Chrome (for web) or Android/iOS tooling if needed
- Access to the Supabase project (URL + anon key)
- Git

Check Flutter:

```
flutter doctor
```

1. Get the code

```
cd "D:\Android Studio Project\attendance_app"
flutter pub get
```

2. Configure Supabase

Config lives in:

```
lib/config/app_config.dart
```

It supports:

1. Compile-time env:

- SUPABASE_URL
- SUPABASE_ANON_KEY

2. Fallback defaults inside `app_config.dart` when env is empty

Local run with defaults

If defaults already point to the correct project, just run:

```
flutter run -d chrome
```

Local run with explicit env

```
flutter run -d chrome \  
  --dart-define=SUPABASE_URL=https://YOUR_PROJECT.supabase.co \  
  --dart-define=SUPABASE_ANON_KEY=YOUR_ANON_KEY
```

Optional for password-reset redirects:

```
--dart-define=APP_ORIGIN=http://localhost:YOUR_PORT
```

3. Run the app

```
flutter run -d chrome
```

Other devices:

```
flutter devices  
flutter run -d windows  
flutter run -d edge
```

After adding packages (e.g. `flutter_map`), do a **full restart**, not only hot reload.

4. First admin user

1. Register a user in the app
2. In Supabase SQL Editor:

```
UPDATE public.users  
SET role = 'admin'  
WHERE email = 'your-admin@email.com';
```

3. Sign out and sign in again
-

5. Useful commands

```
flutter pub get
flutter analyze
flutter test
dart analyze lib
```

6. Common first-day issues

Problem	Fix
Blank / config error	Set Supabase URL + anon key
Signup fails	Run auth SQL bundles / triggers
Password reset lands on login only	Fix Auth Redirect URLs (see deploy doc)
Workplace location missing table	Run supabase_migration_work_site_geofence.sql
Map picker needs restart	Full restart after flutter_map install

Next

- [Supabase and database](#)
- [Architecture](#)

Supabase and database

This page explains how the backend is set up.

Where SQL lives

Location	Use when
supabase_bundles/	New empty Supabase project (run in order)
Root supabase_migration_*.sql	Existing project feature patches

Always read:

- [supabase_bundles/README.md](#)

New project — run order

In Supabase → **SQL Editor**, run:

Step	File
1	01_core_setup.sql
2	02_auth_and_users.sql

3	03_leave.sql
4	04_announcements.sql
5	05_claims.sql
6	06_payroll.sql
7	07_indexes.sql
8	09_work_site.sql (geofence)
Optional	08_seed_payroll_test_data_OPTIONAL.sql

Wait for success before the next file.

Existing project — important migrations

Examples (root folder):

File	Purpose
supabase_migration_work_site_geofence.sql	One-site geofence + updates clock_in_if_allowed
supabase_migration_payroll_rls_fix_recursion.sql	Fixes payslip RLS recursion (42P17)
Username-related migrations	Flexible usernames / login helpers

If a feature “exists in the app but fails in production”, check whether the matching SQL was applied on **that** Supabase project.

Auth essentials

Email provider

Supabase → Authentication → Providers → Email enabled.

URL configuration

Supabase → Authentication → URL configuration:

- **Site URL** = production app URL
Example: `https://attendance-app-peach-rho.vercel.app`
- **Redirect URLs** must include:
 - Production `https://your-app.vercel.app/**`
 - Local `http://localhost:PORT/**` and/or `http://127.0.0.1:PORT/**`

Password reset emails use `redirectTo` with `?passwordReset=1`
(see `lib/utils/auth_redirect.dart`).

Confirm email

For testing, many teams disable “Confirm email”.

For production, decide with the client and test the full flow.

Core tables (high level)

Table / area	Purpose
auth.users	Supabase Auth accounts
public.users	App profile (role, username, HR fields)
attendance	Clock records
leave_requests (+ attachments)	Leave
expense_claims (+ attachments)	Claims
Payroll tables	Runs, items, salary, statutory
company_announcements	News
work_site	Singleton geofence (id = 1)
app_notifications	In-app notifications

Important RPCs / triggers

Name	Purpose
handle_new_user	Creates public.users row on signup (needs username metadata)
get_email_for_login	Username → email for login
is_username_available	Username check
clock_in_if_allowed	Clock-in with leave + geofence checks
distance_meters	Haversine helper for geofence

Storage buckets

Confirm these exist:

- leave-attachments
- claim-attachments

Created by claims/leave SQL bundles (or manually if missing).

Promote admin

```
UPDATE public.users
SET role = 'admin'
WHERE email = 'hr@company.com';
```

Geofence notes for developers

- Table: `public.work_site` `singleton ( id = 1 )`
 - When `is_active = true`, clock-in requires `location` as `"lat,lng"`
 - Server rejects out-of-range punches even if the client is bypassed
 - Admin UI: `lib/screens/admin/admin_work_site_screen.dart`
 - Map picker: OpenStreetMap via `flutter_map`
-

Next

- [Architecture](#)
 - [Features and key files](#)
-

Architecture

How the Flutter app is structured.

High-level flow

```
main.dart
→ Supabase.initialize
→ AuthProvider.init
→ AuthGate
  → LoginScreen (guest)
  → SetNewPasswordScreen (password recovery)
  → AdminShell / EmployeeShell (signed in)
```

State management

- **Provider** with `AuthProvider`
- Profile cached via `SessionProfileCache` for faster startup
- Feature screens mostly use local `StatefulWidget` state + `SupabaseService`

Do **not** assume Riverpod/Bloc — this project uses Provider.

Navigation

File: `lib/utils/app_route.dart`

Platform	Behaviour
Web	Instant push/pop (<code>_InstantPageRoute</code>) — avoids sluggish animated transitions
iOS native	Instant push, short Cupertino reverse
Android / desktop	Short fade

Helpers:

- `pushAppPage(context, page)`
- `popApp(context)`

Prefer these over raw `MaterialPageRoute` for consistency.

*Note: On web/PWA, **browser edge-swipe back** uses History API and can feel slower than the AppBar back button. This is expected with the current route strategy.*

Shells (tabs)

Employee (`EmployeeShell`)

Tabs:

0. Home (`EmployeeHomeTab`)
1. Clock (`EmployeeAttendanceTab`)
2. Profile (`ProfileTab`)

Uses lazy `IndexedStack` so visited tabs stay warm.

Admin (`AdminShell`)

Tabs:

0. Home (`AdminHomeTab`)
1. Attendance overview
2. Employees list

Same lazy `IndexedStack` pattern.

Services

Central API layer:

```
lib/services/supabase_service.dart
```

Also:

Service	Role
<code>app_realtime.dart</code>	Realtime channel helpers
<code>payroll_engine.dart</code>	Payroll calculation
<code>announcement_badge_service.dart</code>	Unread announcement counts
PDF helpers	Profile / payslip export

Prefer adding new backend calls in services — not deep inside widgets.

Time zone

Malaysia time helpers:

```
lib/utils/app_time.dart
```

Attendance “today” and many SQL functions use **Asia/Kuala_Lumpur**.

Auth / password recovery

Piece	File
Reset email + redirect	AuthRedirect, SupabaseService.sendPasswordResetEmail
Recovery flag	AuthProvider.passwordRecoveryPending
Bootstrap from URL	AuthLinkBootstrap (before URL cleanup)
Set password UI	SetNewPasswordScreen
Config contact	help_support_config.dart

Flow:

1. Forgot password sends email with `redirectTo` including `passwordReset=1`
 2. App detects recovery session
 3. Shows set password screen
 4. Updates password, signs out, returns to login with success banner
-

Geofence architecture

```
Admin saves work_site (id=1)
      ↓
Employee clock-in
      ↓
Client: GPS + distance check (fast UX)
      ↓
RPC clock_in_if_allowed (server truth)
```

Utils: `lib/utils/geofence.dart`

Model: `lib/models/work_site.dart`

Performance patterns already used

- In-flight request dedupe / short caches (employees, work site)
- `AsyncLoadGuard` on loads
- `ValueNotifier` for clocks (avoid full rebuild every second)
- Home dashboards: count queries + `RepaintBoundary` + skip unchanged `setState`
- Lazy tab creation in shells

When adding features, avoid:

- Fetching huge lists only to show a count
 - Soft multi-layer shadows on long scrollable grids (web cost)
 - Rebuilding whole dashboards on every realtime event when data did not change
-

Next

- [Features and key files](#)

Features and key files

Use this as a map when you need to change a feature quickly.

Auth & onboarding

Feature	Key files
Login	lib/screens/login_screen.dart
Register	lib/screens/register_screen.dart
Forgot password	lib/screens/forgot_password_screen.dart
Set new password (email link)	lib/screens/set_new_password_screen.dart
Auth state	lib/providers/auth_provider.dart
Redirect / recovery URL	lib/utils/auth_redirect.dart, lib/utils/auth_link_bootstrap.dart
App entry / AuthGate	lib/main.dart
Config	lib/config/app_config.dart

Employee app

Feature	Key files
Shell / tabs	lib/screens/employee/employee_shell.dart
Home	lib/screens/employee/employee_home_tab.dart
Clock in/out	lib/screens/employee/employee_attendance_tab.dart
Attendance history/log	attendance_history_screen.dart, employee_attendance_log_screen.dart
Leave	leave_tab.dart, apply_leave_screen.dart
Claims	claims_screen.dart, submit_claim_screen.dart, claim_detail_screen.dart
Payslips	employee_payroll_history_screen.dart, employee_payslip_detail_screen.dart
Profile	profile_tab.dart
Announcements	lib/screens/announcements_screen.dart
Help	lib/screens/help_support_screen.dart

Admin app

Feature	Key files
Shell / tabs	lib/screens/admin/admin_shell.dart
Home	lib/screens/admin/admin_home_tab.dart
Employees list/edit	employee_list_screen.dart, admin_employee_edit_screen.dart
Attendance overview	attendance_overview_screen.dart, calendar screens
Leave hub	admin_leave_hub_screen.dart, leave_management_screen.dart
Claims	claim_management_screen.dart
Announcements	admin_announcements_screen.dart
Workplace geofence	admin_work_site_screen.dart, work_site_map_picker_screen.dart
OSM map widget	lib/widgets/work_site_osm_map.dart
Payroll hub	lib/screens/admin/payroll/*

Shared services / utils

Concern	File
Almost all backend calls	lib/services/supabase_service.dart
Realtime subscriptions	lib/services/app_realtime.dart
Payroll math	lib/services/payroll_engine.dart + lib/payroll/*
Malaysia time	lib/utils/app_time.dart
Friendly errors	lib/utils/error_messages.dart
Geofence math	lib/utils/geofence.dart
Profile validators	lib/utils/profile_validators.dart
Theme / colours	lib/constants/app_theme.dart
Support phone/email	lib/constants/help_support_config.dart

Models (examples)

Under lib/models/ :

- app_user.dart
- attendance.dart
- leave_request.dart
- expense_claim.dart

- `work_site.dart`
 - payroll models (`payroll_run.dart` , `payroll_item.dart` , ...)
-

SQL feature mapping

App feature	SQL starting point
Users / auth profile	<code>02_auth_and_users.sql</code>
Attendance + clock RPC	<code>01_core_setup.sql</code> , updated in leave/geofence migrations
Leave	<code>03_leave.sql</code>
Announcements	<code>04_announcements.sql</code>
Claims	<code>05_claims.sql</code>
Payroll	<code>06_payroll.sql</code>
Indexes	<code>07_indexes.sql</code>
Geofence	<code>09_work_site.sql</code> OR <code>root supabase_migration_work_site_geofence.sql</code>

Dependencies worth knowing

From `pubspec.yaml` :

- `supabase_flutter`
 - `provider`
 - `geolocator`
 - `flutter_map` + `latlong2` (OpenStreetMap picker)
 - `intl` , `file_picker` , `pdf` , `table_calendar` , `google_fonts` , ...
-

Next

- [Deploy and auth](#)
 - [Handover checklist](#)
-

Deploy and auth URLs

How production web hosting and Supabase Auth redirects fit together.

Typical production URL

Example used in this project:

```
https://attendance-app-peach-rho.vercel.app
```

Update this doc if the production domain changes.

Vercel (or similar) env vars

Set for the Flutter web build:

Variable	Purpose
SUPABASE_URL	Supabase project URL
SUPABASE_ANON_KEY	Supabase anon/public key

Pass them into the build as `--dart-define=...` in your CI/build script.

If env vars are missing/empty, the app falls back to defaults in `lib/config/app_config.dart`.

After changing env vars → **redeploy**.

Supabase Auth URL configuration

Dashboard → **Authentication** → **URL configuration**

Site URL

```
https://attendance-app-peach-rho.vercel.app
```

(No trailing path required; use your real production origin.)

Redirect URLs (examples)

```
https://attendance-app-peach-rho.vercel.app/**
http://localhost:4840/**
http://127.0.0.1:4840/**
```

Add any other local Flutter web ports you use (`flutter run` often changes ports).

Wildcard form `http://localhost:**` may not always be accepted depending on Supabase UI — prefer explicit ports or `/**` patterns Supabase allows.

Password reset redirect

App code builds redirect like:

```
{APP_ORIGIN}?passwordReset=1
```

See:

- `lib/utils/auth_redirect.dart`
- `AuthProvider.sendPasswordResetEmail`
- `SetNewPasswordScreen`

If redirect URL is not allow-listed, users click the email and only see login — recovery UI never opens.

Help & support contact

Edit:

lib/constants/help_support_config.dart

Field	Current idea
supportPhone	+60 11-7078 7014
supportEmail	empty (hidden)
officeHours	Mon–Fri 9:00–18:00 MYT

Storage

Production Supabase project must have buckets:

- leave-attachments
- claim-attachments

Smoke test after deploy

1. Open production URL
2. Sign in as employee → clock tab loads
3. Sign in as admin → home shortcuts load
4. Forgot password → email link opens **Set new password**
5. Admin Workplace location → map loads → save works
6. Employee clock-in with geofence ON near the pin

Next

- [Handover checklist](#)

Handover checklist

Use this on day 1 of taking over the project.

Access you should receive

- ☐ Git repository access
 - ☐ Supabase project access (Dashboard)
 - ☐ Vercel (or host) project access
 - ☐ Production app URL
 - ☐ At least one admin test account
 - ☐ Support phone ownership / update rights (`help_support_config.dart`)
-

Verify the environment

- ☐ `flutter pub get` works
 - ☐ `flutter run -d chrome` opens the app
 - ☐ App talks to the correct Supabase project
 - ☐ SQL bundles / migrations already applied on that project
 - ☐ Storage buckets exist
 - ☐ Auth Site URL + Redirect URLs match production + local
-

Verify product flows

Employee

- ☐ Register / login
- ☐ Forgot password → set new password → login
- ☐ Clock in / clock out
- ☐ Apply leave
- ☐ Submit claim (MYR)
- ☐ View announcements
- ☐ Open Help & support

Admin

- ☐ Home dashboard loads
 - ☐ Edit an employee
 - ☐ Approve/reject leave
 - ☐ Approve/reject claim
 - ☐ Open payroll hub
 - ☐ Post announcement
 - ☐ Set Workplace location on map, enforce ON, save
 - ☐ Confirm employee outside radius cannot clock in
-

Read these docs

- ☐ [docs/README.md](#)
 - ☐ [User getting started](#)
 - ☐ [Employee guide](#)
 - ☐ [Admin guide](#)
 - ☐ [Developer overview](#)
 - ☐ [Setup and run](#)
 - ☐ [Supabase and database](#)
 - ☐ [Architecture](#)
 - ☐ [Features and key files](#)
 - ☐ [Deploy and auth](#)
-

Known care points (do not ignore)

1. **Wrong Supabase project** — many “bugs” are missing SQL on the active project
 2. **Password reset Redirect URLs** — must include app origin
 3. **Geofence SQL** — table `work_site` + updated `clock_in_if_allowed` required
 4. **Payroll RLS recursion** — run fix migration if payslips error 42P17
 5. **Web performance** — avoid heavy rebuilds/shadow stacks on dashboards
 6. **Signup rate limit (429)** — tell testers to wait, not spam Create account
 7. **OSM tiles** — `flutter_map` uses OpenStreetMap; respect tile usage policy for heavy production traffic
-

When you change a feature

1. Update the matching **user** doc if staff/admin steps change
 2. Update **developer** feature map if files move
 3. Add/adjust SQL migration if schema/RPC changes
 4. Redeploy web + confirm Auth URLs if domain changes
-

Support config quick edit

`lib/constants/help_support_config.dart`

- Phone, email, office hours shown in Help & support
-

You're ready when the checkboxes above are done and you can demo employee + admin happy paths without guessing.